Project 2 : Choosing a sparse solver

Due Monday, November 9

1 Overview

Project 1 was one algorithm, written from scratch, and the work was making it fast. This project is the opposite. You will write almost no numerical code yourself. Instead, you will pick up tools other people have already built and figure out which one is right for the job. You will take one family of sparse symmetric positive definite (SPD) systems and attack it two ways:

1. **Direct:** sparse Cholesky factorization, under different fill-reducing orderings.
2. **Iterative:** conjugate gradient, under different preconditioners.

You will run experiments, figure out which methods win, when, and why, and then encode what you learned in two functions, `solve2d` and `solve3d`, each using the best method you found for its kind of system.

This project is deliberately open-ended. There is no single correct answer to the questions in Section 4. I will grade "correctness" on whether your experiments are sound and whether your conclusions follow.

2 The problem

Imagine groundwater seeping through a block of rock. The rock is not uniform: some layers are open sandstone and some are nearly impermeable shale, and the permeability k varies by four orders of magnitude from place to place. The steady-state water pressure u satisfies the diffusion equation

$$-\nabla \cdot \big(k(\boldsymbol{z})\, \nabla u(\boldsymbol{z})\big) = f(\boldsymbol{z})$$

on the unit square ($d = 2$) or unit cube ($d = 3$), with $u = 0$ on the boundary. You do not need to know anything about this equation. If we chop the domain into a grid of m cells per side, put one unknown in each cell, and balance the flow between each cell and its neighbors, then the result is a linear system $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ with $n = m^d$ unknowns, where

- each row of $\boldsymbol{A}$ couples a cell to itself and its $2d$ nearest neighbors, so $\boldsymbol{A}$ has at most 5 (in 2D) or 7 (in 3D) nonzeros per row;
- $\boldsymbol{A}$ is symmetric positive definite;
- the entries of $\boldsymbol{A}$ vary wildly in size from row to row, because k does.

You are given a file `problem.jl` that builds the system for you:

```
include("problem.jl")
A, b = problem(200; d = 2)   # 2D grid, 200 x 200 cells, n = 40,000
A, b = problem(40;  d = 3)   # 3D grid, 40 x 40 x 40 cells, n = 64,000
```

`A` is a `SparseMatrixCSC{Float64, Int}` and `b` is a `Vector{Float64}`. Cells are numbered in the natural order: the first coordinate varies fastest, exactly like the column-major layout of an $m \times m$ (or $m \times m \times m$) array.

3 Requirements

3.1 The `solve2d` and `solve3d` functions

1. You must submit a single `.jl` file containing the definitions of two functions, `solve2d` and `solve3d`. I will only call `solve2d` on 2D systems and `solve3d` on 3D systems. Your file may (and should) contain other functions that they call.

2. Each function must take an SPD sparse matrix `A` and a vector `b`, and a keyword argument `rtol`, and return a vector `x` satisfying

$$\frac{\|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}\|_2}{\|\boldsymbol{b}\|_2} \leq \texttt{rtol}.$$

   This is the *true* residual of the system you were given. If you reorder, scale, or precondition, it is still this quantity that must be small when you return. The skeleton is

```
function solve2d(A::SparseMatrixCSC{Float64,Int}, b::Vector{Float64};
                 rtol = 1e-8)
    # Function body here
    return x
end

function solve3d(A::SparseMatrixCSC{Float64,Int}, b::Vector{Float64};
                 rtol = 1e-8)
    # Function body here
    return x
end
```

3. Your functions must not modify `A` or `b`.

3.2 What you may use

Anything. Use any registered Julia package, and combine them however you like. You'll need to use `SparseArrays` for access to the `SparseMatrixCSC` objects. For more packages, some good places to start are:

- **Direct methods:** The sparse Cholesky, `cholesky(A)` from `LinearAlgebra`. You can hand it an ordering with `cholesky(A; perm = p)`, where `p` is a permutation vector. For orderings, try `AMD.jl`, `Metis.jl`, and `SymRCM.jl`.

- **Iterative methods:** the conjugate gradient routines in `Krylov.jl` or `IterativeSolvers.jl`. **Note:** Both of these packages make the mistake (in my opinion) of interleaving multi-threaded BLAS Level-1 calls (such as `dot`) with single-threaded `(SparseMatrixCSC) * (AbstractVector)` calls, which can cause serious slow-downs. For your iterative methods, try restricting BLAS to one thread with `BLAS.set_num_threads(1)` (from `LinearAlgebra`), right before the CG call. Be aware that this setting is global: it stays in effect after your function returns, and it applies to everything else, including `cholesky`. Set the thread count explicitly before each method you time, and have your functions put it back the way they found it. To read the current value, use `BLAS.get_num_threads()`.
- **Preconditioners:** you will need to read the documentation to find out what each of these actually computes: `Preconditioners.jl`, `AlgebraicMultigrid.jl`, `IncompleteLU.jl`, and `LimitedLDLFactorizations.jl`.

One warning. Not every CG package measures convergence the same way. Some of them, once you hand them a preconditioner, stop when a *preconditioned* residual is small, which is not the residual in Requirement 2. Find out what your package checks before you trust its `rtol`.

4 The report

Along with your code, submit a report of at most four pages, *including figures*, that answers the questions below. Each answer should be backed by at least one figure or table from your own experiments. Explore both $d = 2$ and $d = 3$, and a range of m wide enough to see trends. Plot scaling results on log-log axes.

1. **Orderings.** For sparse Cholesky, compare at least four orderings: the natural ordering, reverse Cuthill–McKee (RCM), approximate minimum degree (AMD), and nested dissection (METIS). Also find out which ordering `cholesky(A)` uses when you don't give it one. Measure the fill (`nnz` of $\boldsymbol{L}$ against `nnz` of $\boldsymbol{A}$) and the time to factor. Which ordering works best? Does the answer depend on d? Is the ordering with the least fill always the fastest?
2. **Preconditioners.** Compare CG with no preconditioner, Jacobi (diagonal), an incomplete Cholesky factorization, and at least one other preconditioner of your own choosing. For each, measure the number of iterations and the total time to solve, including any setup. Which preconditioner converges in the fewest iterations? Which is fastest overall? If those are different, explain why. How does the iteration count grow with n?
3. **Direct versus iterative.** For each d, at what problem size does your best iterative method overtake your best direct method, if it ever does? How does the answer move as you loosen or tighten `rtol`? Explain the crossover using what you measured in the first two questions, not just by reporting it.
4. **Your solvers.** Describe the method each of `solve2d` and `solve3d` uses, and the evidence from questions 1–3 that justifies each choice.

Finally, in a paragraph or less, tell me one thing that surprised you.

5 Testing

You are given three files: `problem.jl`, `test.jl`, and `project2.sbatch`. Program `test.jl` is how your submission will be tested, and `project2.sbatch` is how this test is submitted to the cluster.

5.1 test.jl

Program `test.jl` loads your script as a module, builds a system with `problem.jl`, calls your `solve2d` or `solve3d` (whichever matches the system), checks the residual, and then times it. The time includes *everything* your function does: choosing an ordering, factoring, building a preconditioner, and iterating.

Point `test.jl` at your submission by editing `FILE_PATH` near the top of `test.jl`:

```
FILE_PATH = "my_sooper_dooper_solver.jl"
module Submission
    include(Main.FILE_PATH)
end
```

You can run `test.jl` for a given dimension, grid size, and tolerance from a command line:

```
    julia test.jl -d 3 -m 40 --rtol 1e-6
```

An example of a successful output might be:

```
    [ Info: Testing my_sooper_dooper_solver.jl
    [ Info: Running solve3d with d=3, m=40 (n=64000), rtol=1.0e-6
    [ Info: Julia threads: 1, BLAS threads: 48
    [ Info: Your solution meets the tolerance (relative residual 8.2e-7).
    [ Info: Benchmarking...
    84.120 ms (412 allocations: 31.27 MiB)
```

For your experiments you will want your own scripts that loop over sizes and methods and record the results. That is expected and encouraged. But `test.jl` is how I will test and grade your functions. Remember, **do not run anything on the login node of the HPC.**

5.2 project2.sbatch

File `project2.sbatch` submits the above test to a `compute` node on the HPC with a fixed amount of resources:

```
    #!/bin/bash
    # ------------------------------------------------------------------
    # Project 2
    # ------------------------------------------------------------------
    #SBATCH --job-name=project2
    #SBATCH --output=project2-%j.out
    #SBATCH --time=00:30:00
    #SBATCH --nodes=1
    #SBATCH --ntasks=1
    #SBATCH --cpus-per-task=48
    #SBATCH --mem=16G
    #SBATCH --partition=compute

    module load julia

    julia test.jl -d 3 -m 40 --rtol 1e-6
```

Sparse factorizations can use a *lot* of memory, especially in 3D and especially under a bad ordering. If your job dies with an out-of-memory error, that is not (necessarily) a bug in your code: it is a data point for question 1. Record it and move on. Stay within $m \leq 1000$ in 2D and $m \leq 80$ in 3D. Some methods will also be very slow at the top of those ranges; you don't need to run every method at every size.

6 Grading

You will be graded 90% on correctness, 10% on efficiency, and a possible +10% bonus for extreme efficiency.

6.1 Correctness (90%)

Correctness has two parts, and you need both:

- `solve2d` and `solve3d` return an `x` meeting the tolerance, for any m and `rtol` I choose. One way this can be verified is `test.jl` outputting

```
[ Info: Your solution meets the tolerance (relative residual 8.2e-7).
```

- Your report answers all four questions in Section 4, with each answer supported by your own data.

6.2 Efficiency (10%, plus 10% bonus)

I will time `solve2d` on a 2D system and `solve3d` on a 3D system. Each system has its own baseline and its own roofline, so there are two of each, listed in the table below.

You will receive 10% for efficiency if `solve2d` is faster than the 2D baseline and `solve3d` is faster than the 3D baseline, each plus a small safety buffer in your favor.

You will receive an additional 10% bonus if `solve2d` is faster than the 2D roofline and `solve3d` is faster than the 3D roofline, again each plus a small safety buffer in your favor. Each roofline is the best of the methods I tried for that system.

	system	baseline	roofline
2D	`-d 2 -m 500 --rtol 1e-8`	**4.5 s**	**0.5 s**
3D	`-d 3 -m 80 --rtol 1e-6`	**3.5 s**	**2.0 s**

I will time your functions by submitting `test.jl` through `project2.sbatch`.

7 Submission

Upload to D2L the `.jl` file containing `solve2d`, `solve3d`, and their dependencies, and your report as a PDF. Name both files after yourself, all lower-case, with underscores between your first and last name, e.g. `paul_may.jl` and `paul_may.pdf`.